

Análisis de Complejidad y Comparativa de Algoritmos de Ordenamiento

Programación Competitiva - Semana 5

Jhon Daniel Pari Valencia

1. Descripción del Problema

Para una programación eficiente tenemos que tomar en cuenta el tiempo de ejecución para cada problema, donde algunos son más rápidos para cierta cantidad de datos y otros para datos masivos. En este documento se presenta la comparación entre los algoritmos de ordenamiento para ver el progreso y comportamiento con la misma entrada para los diferentes algoritmos.

2. Algoritmos Implementados

Se han seleccionado cuatro algoritmos con diferentes complejidades computacionales para observar la brecha de rendimiento:

- **Bubble Sort** ($O(n^2)$): Algoritmo elemental basado en intercambios sucesivos de elementos adyacentes.
- **Selection Sort** ($O(n^2)$): Ordenamiento por selección que minimiza el número de intercambios al buscar el valor mínimo en cada iteración.
- **Insertion Sort** ($O(n^2)$): Algoritmo que construye el arreglo final un elemento a la vez, siendo altamente eficiente para conjuntos de datos pequeños o parcialmente ordenados.
- **Merge Sort** ($O(n \log n)$): Algoritmo basado en la técnica de *Divide y Vencerás*. Es el más robusto de la lista, garantizando un rendimiento óptimo incluso en el peor de los casos.

3. Guía de Compilación y Ejecución

Para compilar y ejecutar el código en sistemas Linux (Fedora o Pop!_OS), se deben seguir estos pasos desde la terminal:

3.1. Compilación

Se recomienda el uso de niveles de optimización para obtener mediciones de tiempo más precisas:

```
g++ -O3 algordenamientos.cpp -o algordenamientos
```

3.2. Ejecución

```
./algordenamientos
```

4. Resultados y Estadísticas

Al finalizar la ejecución, el programa presenta una matriz de comparación ordenada de menor a mayor tiempo, permitiendo visualizar rápidamente el ranking de eficiencia.

Pos	Algoritmo	Pasos	Tiempo (ms)
1	Merge Sort	8715	6800 ms
2	Selection Sort	500499	530000 ms
3	Bubble Sort	743645	933730 ms

Figura 1: Captura de resultados 1

```
PS D:\PROGRAMACION COMPETITIVA\semana 5\ordenamientos> ./algoritmos
Cantidad de numeros: 100
Rango (min max): 3
500

Array Generado: 44 44 361 109 248 289 27 477 73 65 230 260 376 396 4 494 10 491 348 459 24 165 419 156 295 433 492 293 299 476 470 315 332 87 3
78 493 272 406 106 435 320 408 449 456 246 185 204 244 368 397 152 257 295 390 120 412 260 282 142 283 367 50 281 351 291 229 79 479 343 241 59
402 455 258 469 413 69 228 492 65 218 176 183 37 333 389 487 407 488 415 15 72 227 370 62 127 185 93 24 110

Pos Algoritmo Pasos Tiempo (ns)
1 Selection Sort 5049 69700 ns
2 Merge Sort 537 110400 ns
3 Bubble Sort 7560 169900 ns

PS D:\PROGRAMACION COMPETITIVA\semana 5\ordenamientos>
```

Figura 2: Captura de resultados 2

Enlace al Repositorio Principal:

<https://github.com/DANIEL232518/Programacion-Competitiva-2026/tree/main/semana%205/ordenamientos>